Online Training Program
- ASP.NET Core Web API Online Training

ASP.NET WEB API – Basics
- Introduction to ASP.NET Web API
- Creating ASP.NET Web API Application
- How to add Swagger in Web API
- How to use Fiddler to test Web API
- How to use POSTMAN to test Web API
- ASP.NET Web API using SQL Server
- Content Negotiation in Web API
- Media Type Formatter in Web API
- How to Implement GET Method in Web API
- How to Implement POST Method in Web API
- How to Implement PUT Method in Web API
- How to Implement DELETE Method in Web API
- Custom Method Names in Web API
- Parameter Binding in Web API
- Consuming Web API Service From jQuery
- Calling Web API Service in a Cross Domain Using jQuery AJAX
- Cross-Origin Resource Sharing in WEB API

WEB API Routing and Attribute Routing
- ASP.NET Web API Routing
- Routing Variations in WEB API
- Routing and Action Selection in Web API
- ASP.NET Web API Attribute Routing
- Optional Parameters in Web API Attribute Routing
- Route Prefix in Web API
- Web API Attribute Routing Route Constraints
- Route Names and Route Orders in Attribute Routing

ASP.NET WEB API – Security
- Authentication and Authorization in ASP.NET Web API
- ASP.NET Web API Basic Authentication
- Role-Based Basic Authentication in Web API
- Web API Service with Basic Authentication
- HTTP Message Handlers in WEB API
- Basic Authentication Using Message Handler in Web API
- HTTP Client Message Handler in Web API
- Token Based Authentication in ASP.NET Web API
- Client Validation Using Basic Authentication in Web API
- Refresh Token in Web API
- Consume Refresh Token in C# Client
- HMAC Authentication in Web API

Web API Versioning
- Web API Versioning Using URI
- Web API Versioning using Query string Parameter
- Web API Versioning Using Custom Header
- Web API Versioning Using Accept Header
- Web API Versioning Using Custom Media Types

Advanced Concepts
- Enable SSL in Visual Studio Development Server
- Enable HTTPS in Web API

Popular ASP.NET Web API Books
- Most Recommended ASP.NET Web API Books

ASP.NET Web API Attribute Routing

Back to ASP.NET Web API Tutorials For Beginners and Professionals

ASP.NET Web API Attribute Routing

In this article, I am going to discuss **ASP.NET Web API Attribute Routing** with examples. As we already discussed in the **ASP.NET Web API Routing** article that the Routing is a pattern matching mechanism to match an incoming HTTP Request to an action method of a controller.

Attribute Routing in ASP.NET Web API

The ASP.NET Web API 2 and ASP.NET MVC 5 supports a new type of routing called **attribute routing**. As the name implies, attribute routing means attributes are used to define routes. The Attribute routing provides more control over the URIs in your Web API application by defining routes directly on the actions and controllers. For example, you can easily create URIs that describes the hierarchies of resources.

The earlier style of routing called convention-based routing is still fully supported by ASP.NET Web API. In fact, you can combine both approaches in the same project. In this article, we will discuss how to enable attribute routing in Web API and describes the various options for attribute routing.

Why do we need Web API Attribute Routing?

The first release of Web API uses the convention-based routing. In convention-based, we can define one or more route templates in the WebApiConfig file, which are basically parameterized strings. When the Web API Framework receives an HTTP request, it matches the URI against the route template that is available in the Route Table. For more information about convention-based routing, Please read the following articles where we discussed the Convention based **Routing in Web API** with examples.

One advantage of convention-based routing is that all the URI templates are defined in a single place, and the routing rules are applied consistently across all the controllers.

But the convention-based routing in ASP.NET Web API makes it hard to support certain URI patterns that are common in RESTful APIs. For example, resources often contain child resources. Customers have orders, movies have actors, books have authors, etc. It's natural to create URIs that reflects these relations:

Let's understand this with an example.

Step1: Create a new Web API application. Name it AttributeRoutingInWEBAPI

Step2: Right-click on the "Models" folder and add a class file with the name Student.cs and then copy and paste the following code.

```
namespace AttributeRoutingInWEBAPI.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; }
    }
}
```

Step3: Now, add Students Controller.

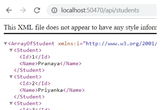
To do so Right-click on the Controllers folder and add a new **Web API2 controller – Empty**. Name it StudentsController.cs. Copy and paste the following code.

```
namespace AttributeRoutingInWEBAPI.Controllers
{
    public class StudentsController : ApiController
    {
        static List<Student> students = new List<Student>()
        {
            new Student() { Id = 1, Name = "Pranaya " },
            new Student() { Id = 2, Name = "Priyanka" },
            new Student() { Id = 3, Name = "Anurag" },
            new Student() { Id = 4, Name = "Sambit " }
        };

        public IEnumerable<Student> Get()
        {
            return students;
        }

        public Student Get(int id)
        {
            return students.FirstOrDefault(s => s.Id == id);
        }

        public IEnumerable<string> GetStudentCourses(int id)
        {
            List<string> CourseList = new List<string>();

            if (id == 1)
                CourseList = new List<string>() { "ASP.NET", "C#.NET", "SQL Server" };
            else if (id == 2)
                CourseList = new List<string>() { "ASP.NET MVC", "C#.NET", "ADO.NET" };
            else if (id == 3)
                CourseList = new List<string>() { "ASP.NET WEB API", "C#.NET", "Entity
Framework" };
            else
                CourseList = new List<string>() { "Bootstrap", "jQuery", "AngularJs" };

            return CourseList;
        }
    }
}
```

In Web API1, we had the convention-based routing that defines the routes using the route templates. When we create a new Web API project the Web API Framework creates a default route in the **WebApiConfig.cs** file. The default route is shown below

```
public static class WebApiConfig
{
    public static void Register(HttpConfiguration config)
    {
        // Web API routes
        config.MapHttpAttributeRoutes();

        config.Routes.MapHttpRoute(
            name: "DefaultApi",
            routeTemplate: "api/{controller}/{id}",
            defaults: new { id = RouteParameter.Optional }
        );
    }
}
```

So with the above default route and the **StudentsController** in place (**/api/students** is mapped to the **Get()** method of StudentsController as expected as shown in the below image.




```
- <ArrayOfStudent xmlns:i="http://www.w3.org/2001/
  - <Student>
      <Id>1</Id>
      <Name>Pranaya</Name>
    </Student>
  - <Student>
      <Id>2</Id>
      <Name>Priyanka</Name>
    </Student>
  - <Student>
      <Id>3</Id>
      <Name>Anurag</Name>
    </Student>
  - <Student>
      <Id>4</Id>
      <Name>Sambit</Name>
    </Student>
  </ArrayOfStudent>
```

But when we navigate to **/api/students/1** we get the following exception message

Multiple actions were found that match the request: Get on type AttributeRoutingInWEBAPI.Controllers.StudentsController GetStudentCourses on type AttributeRoutingInWEBAPI.Controllers.StudentsController

This is because the Web API Framework does not know which of the 2 following action methods to map to the URI **/api/students/1**

Get(int id) or GetStudentCourses(int id)

This can be very easily achieved by using the **Attribute Routing** in **ASP.NET Web API**. Here is what we want the Web API Framework to do

1. URI **/api/students/1** should be mapped to **Get(int id)**. This method should return the student by id.
2. The URI **/api/students/1/courses** should be mapped to **GetStudentCourses(int id)**. This method should return the student courses by student id.

To achieve the above, we need to decorate the GetStudentCourses() action method with the [Route] attribute as shown in the below image

```
[Route("api/students/{id}/courses")]
public IEnumerable<string> GetStudentCourses(int id)
{
    List<string> CourseList = new List<string>();

    if (id == 1)
        CourseList = new List<string>() { "ASP.NET", "C#.NET", "SQL Server" };
    else if (id == 2)
        CourseList = new List<string>() { "ASP.NET MVC", "C#.NET", "ADO.NET" };
    else if (id == 3)
        CourseList = new List<string>() { "ASP.NET WEB API", "C#.NET", "Entity Framework" };
    else
        CourseList = new List<string>() { "Bootstrap", "jQuery", "AngularJs" };

    return CourseList;
}
```

At this point build the solution and navigate to **/api/students/1**. Notice that, now you will get the student details whose id=1 and when you navigate to **/api/students/1/courses** you will get all the courses into which student with id=1 is enrolled. Let us see some of the examples where attribute routing makes it easy.

API versioning

In the below example, the route "/apiv1/students" would be routed to a different controller than the "/apiv2/students" route.

/apiv1/students

/apiv2/students

Overloaded URI segments

In this example, "1" is an order number, but "pending" maps to a collection.

/orders/1

/orders/pending

Multiple parameter types

In this example, "1" is an order number, but "2013/06/16" specifies a date.

/orders/1

/orders/2013/06/16

How to enable Web API Attribute Routing?

In Web API 2, the Attribute Routing is enabled by default. The **config.MapHttpAttributeRoutes();** code which is present in **WebApiConfig.cs** file enables the Web API Attribute Routing. We can also combine the Web API Attribute Routing with convention-based routing. To define convention-based routes, call the **MapHttpRoute** method as shown below.

```
public static class WebApiConfig
{
    public static void Register(HttpConfiguration config)
    {
        // Attribute routing.
        config.MapHttpAttributeRoutes();

        // Convention-based routing.
        config.Routes.MapHttpRoute(
            name: "DefaultApi",
            routeTemplate: "api/{controller}/{id}",
            defaults: new { id = RouteParameter.Optional }
        );
    }
}
```

Can we use both Attribute Routing and Convention-based routing in a single Web API project?

Yes, We can combine both the routing mechanisms in a single ASP.NET Web API project. The controller action methods that have the [Route] attribute uses the Attribute Routing and the others without [Route] attribute uses Convention-based routing.

Note: You need to configure the Attribute routing before the convention-based routing in ASP.NET Web API.

What are the advantages of using Web API Attribute Routing?

1. It gives us more control over the URIs than convention-based routing. Creating URI patterns like hierarchies of resources (For example, students have courses, Departments have employees) is very difficult with convention-based routing.
2. Reduces the chances for errors, if a route is modified incorrectly in RouteConfig.cs then it may affect the entire application's routing.
3. May decouple controller and action names from route entirely.
4. Easy to map two routes pointing to the same action.

In the next article, we will discuss **Optional Parameters in Web API Attribute Routing** with examples. Here, in this article, I try to explain the **Web API Attribute Routing** step by step with some examples. I hope this ASP.NET Web API Attribute Routing article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this Web API Attribute Routing article.

DOT NET TUTORIALS
Dot Net Tutorials
About the Author: Pranaya Rout
Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name* Email* Website

Post Comment

© Dot Net Tutorials | Website Design by Sunrise Pixel